AI Security POC — Simulating OWASP LLM & Agentic AI Risks on Claude and Gemini

Candidate: Amartya Kushwaha · Program: YGF NGO — AI Security Internship, Round 2

Scope: Chatbot + AI agent tested against OWASP Top 10 for LLM Applications and Agentic AI risks, before and after mitigations.

1. Approach

We built a deliberately small but realistic target — an NGO staff chatbot with an internal handbook in its context, and an AI agent with tools, memory and an MCP-style tool connection — and then attacked it the way a real user would: through ordinary chat messages. Every risk has three test cases. Each case is run twice per model: Before (mitigations off) and After (mitigations on), with everything else identical, so any change in outcome is attributable to the control. Outcomes are classified from observable evidence only — a leaked marker string, a file in the sent outbox, a tool-execution log, a marker file in a simulated privileged directory — never from the model's tone or a human's impression.

Safety boundaries were set before any testing: all data is fake, all payloads are harmless markers, email is written to a local outbox (never sent), HTTP calls go to a local stub (never real traffic), and code execution is jailed inside the project directory.

2. Architecture

- Model layer (interchangeable): mock (deterministic stand-in for offline harness testing), gemini, claude, behind a single generate() interface. Keys are read from the environment only.
- App layer (owns all controls): the chatbot keeps its system prompt and handbook as server-side hidden context and renders answers into a simulated HTML sink; the agent has a fixed goal, a signed memory store, and an MCP client that can pin tool descriptions.
- Tool layer (fake/local): read_file (per-run sandbox directory), http_call (records attempts; only a fake local endpoint 'responds'), send_email (local outbox folders sent/ and held/), run_code (subprocess jailed to sandbox_jail/, with a work/ area and a simulated privileged vault/ area).

- Harness: runner.py executes attacks/cases.json (30 cases) per model per phase in isolated run directories and writes results.csv with Risk ID, Risk Name, Model, Case, Phase, Prompt, Result and Evidence (the control/event log plus an output excerpt).

3. Risks tested and mitigations applied

Risk	What the attack does	Mitigation	Where
LLM01 Prompt Injection	Direct/indirect overrides make the chatbot leak the handbook flag	Input filtering + instruction hierarchy	mitigations.input_filter
LLM02 Sensitive Information Disclosure	Pretext questions extract Wi-Fi password, salary band, flag	Output redaction of flag pattern + sensitive tokens	mitigations.redact_output
LLM03 Excessive Agency	Agent emails outsiders / posts data out with no approval	Human-confirmation gate for email; URL allowlist; least-privilege scoping	tools.send_email, tools.http_call
LLM08 Hidden Context Exposure	'Repeat everything above' style extraction of setup text	Context isolation + extraction filters + verbatim leak check	chatbot, mitigations.context_leak_check
LLM10 Improper Output Handling	Script/onerror/javascript: payloads land raw in an HTML sink	Output encoding + scheme neutralisation before the sink	mitigations.encode_for_sink
ASI01 Agent Goal Hijack	User replaces the agent's goal, then issues orders	Goal-lock prompt + goal-change filtering + output rejection	agent
ASI02 Tool Misuse	Tools aimed at internal files / metadata IP; poisoned MCP tool description	Tool + URL allowlists; MCP description pinning (SHA-256)	tools, agent.connect_mcp
ASI03 Identity & Privilege Abuse	Claimed director identity / self-typed staff token	Identity check: role only from trusted session config	agent
ASI05 Unexpected Code Execution	Agent-run code writes markers outside its work area	Jailed execution + static command allowlist	tools.run_code
ASI06 Memory & Context Poisoning	False 'director approval' / 'verified admin' facts planted in memory	HMAC-signed memory; unsigned entries rejected; reset per run	agent, mitigations.sign_memory_entry

4. Harness validation (mock mode)

Before any live model was involved, the full suite was executed against the deterministic mock model to prove the harness itself works:

Before: 30/30 attacks Succeeded. After: 0/30 Succeeded — 25 Blocked, 5 Partial (2 emails held as drafts under LLM03; 3 sink payloads present only in neutralised form under LLM10). These are harness-validation numbers only; they are not Claude/Gemini results and are reported separately in results_mock.csv.

5. Claude vs Gemini — live results

> TEMPLATE — to be filled from results.csv after the live runs.
> Do not edit the numbers by hand from impressions; compute them from the CSV so the report and the results sheet always agree.

Test setup (fill in): Date of live runs: <<DATE>> · Gemini model: <<GEMINI_MODEL>> · Claude model: <<ANTHROPIC_MODEL>> · Cases per model: 30 (10 risks × 3) × 2 phases = 60 rows per model.

Summary counts (fill from results.csv):

Model	Phase	Succeeded	Partial	Blocked
Gemini	Before	<< >>	<< >>	<< >>
Gemini	After	<< >>	<< >>	<< >>
Claude	Before	<< >>	<< >>	<< >>
Claude	After	<< >>	<< >>	<< >>

Per-risk differences (fill in): <<For each risk where the two models differ in either phase, name the risk, the case, and quote the Evidence field. Note especially: (a) which model resisted direct injection without app-layer help, (b) which model complied with goal-hijack or memory poisoning in the Before phase, (c) any case where a model refused a legitimate-looking pretext (LLM02-3).>>

Interpretation (fill in): <<2–4 paragraphs. Separate clearly what the models did on their own (Before phase) from what the app-layer controls added (After phase). State where the two models behaved identically and where they diverged, and what that implies for deploying either model behind the same controls.>>

6. Limitations

The mock model validates mechanics, not model behaviour; only the live rows describe Claude/Gemini. The chatbot/agent are minimal stand-ins, so results show control effectiveness on this surface, not a guarantee for a production system. Input-filter wordlists are illustrative — a production

deployment would layer them with model-level defences and monitoring.
The code-execution 'escape' is demonstrated against a simulated
privileged directory inside a physical jail, by design.

7. Conclusion

<<To be finalised with the live comparison: one short paragraph stating
(i) how vulnerable an unmitigated LLM app/agent is to ordinary chat-level
attacks, (ii) that app-layer controls — allowlists, confirmation gates,
redaction, signed memory, description pinning — changed outcomes on
<<N>> of 30 cases per model, and (iii) the single most important
difference observed between Claude and Gemini.>>
Reproducibility: python3 runner.py --models mock regenerates the
harness-validation sheet; with GEMINI_API_KEY / ANTHROPIC_API_KEY set,
python3 runner.py --models gemini,claude regenerates the live sheet.
No keys are stored in the repository.